Ben-Gurion University of the Negev
The Stein Faculty of Computer and Information Science
The Institute of Software, Systems and Security (S^3)

Methods for Detecting Attacks

AutoResearch-Inspired Evaluation of Internal-Channel Risks in Agentic Systems

---

Submitted by

Natalie Mirelashvili
322695314

Yuval Aviv
315106948

Ravit Cohen
313188799

Michael Goldfarb
209507763

Lecturer: Prof. Asaf Shabtai

August 12, 2026

Abstract

Large language model (LLM) agents are increasingly augmented with tools, persistent memory, retrieval components, and the ability to communicate with other agents, which extends their capabilities but also broadens their attack surface. This is particularly true for multi-agent systems, where sensitive information can propagate through internal channels - messages between agents, tool arguments, memory writes, and logs before, or even without, reaching the final user-facing output. This project proposes an AutoResearch-inspired red-teaming framework for agentic and multi-agent LLM systems. Attack variants are generated through a heuristic search procedure over a defined attack space, evaluated against a target system on a fixed task, and scored to guide the next iteration. We report a small-scale exploratory evaluation using `llama3.2:3b` through Ollama on four agentic systems, LangGraph, AgentDojo, AutoGen, and CrewAI, across three experimental objectives: information leakage, external leakage into the final output, and performance degradation. Leakage attack success rates ranged from 0.958 to 1.000 across all four systems, and internal-channel leakage was consistently higher than final-output leakage, indicating that final-output-only evaluation can underestimate measured exposure. Automated degradation search did not produce a measurable task-success drop in the reported experiment. A separate diagnostic evaluation indicated that manually specified degradation mechanisms can affect task success, but the automated search did not reproduce that behavior.

1 Introduction

Large language model (LLM)-based agents are increasingly used for tasks that go beyond single-turn question answering: they call external tools and APIs, retrieve and incorporate documents, read and write persistent memory, and plan a sequence of actions toward a goal. In multi-agent systems, several such agents are composed into a single pipeline, for example, a planner that decomposes the task, a retriever or tool-using agent that gathers external information, a worker agent that carries out the core task, and a reviewer or guard agent that checks the result before it is returned to the user.

Although this decomposition improves modularity and often enhances task performance, it also introduces internal communication channels, such as inter-agent messages, tool-call arguments, memory writes, and logs, that are not directly visible to the end user.

Existing evaluations of prompt-injection robustness for LLM-integrated systems typically focus on whether the final, user-facing response leaks sensitive content or deviates from the intended task [1, 2]. In an agentic or multi-agent pipeline, however, a sensitive value can pass through, and be exposed at several internal points before any final answer is produced. An attacker who injects malicious content into a retrieved document, a tool's output, a task input, or a message exchanged between agents may cause a secret to surface in a tool argument, an inter-agent message, or a memory entry, even when the system's final response to the user appears unremarkable. Evaluating only the final output can therefore substantially underestimate how exposed such a system really is, and may also miss attacks that degrade task performance or increase operational cost without leaking anything at all.

This project combines internal-channel-aware evaluation with AutoResearch-style attack discovery by using internal traces as feedback signals to guide an automated search over attack variants. We ask (i) whether this iterative process can identify effective attack variants under a limited search budget, and (ii) whether final-output-only evaluation misses leakage and degradation effects in internal channels. We adopt an AutoResearch-style loop, described below: a heuristic search procedure generates an attack variant, the variant is run against a target agentic system on a fixed task, the outcome is scored against a chosen metric, and the variant is kept or discarded before the next is generated. In the reported evaluation, we operationalize these risks through three experimental goals: general channel-aware leakage, external final-output leakage, and performance degradation. The framework can also amplify operational cost through extra tool calls and token usage; although latency can be recorded as a diagnostic signal, the reported evaluation focuses on task success, leakage, tool-call increase, and operational degradation score rather than latency. We study these goals across four systems: AgentDojo, AutoGen, CrewAI, and LangGraph, configured with a shared abstract pipeline to support compar-

ison across frameworks. Throughout, the attacker only injects content into untrusted channels (retrieved documents, tool outputs, task inputs, inter-agent messages), no real user data is involved and all sensitive information is represented by synthetic canary secrets in sandboxed tasks.

1.1 Agentic and Multi-Agent LLM Systems

An agentic LLM system augments a base language model with the ability to call external tools or APIs, retrieve and incorporate external documents, read and write to a memory store, and plan over multiple steps rather than respond in a single turn. AgentDojo [3] is one such environment: it provides realistic tool-using tasks together with an established benchmark for prompt-injection attacks and defenses, and serves in this project as a single-agent, tool-using reference point.

Multi-agent frameworks compose several agents of this kind into a single pipeline. AutoGen [4] structures this composition as a conversation in which agents exchange messages and may invoke tools or request human input. CrewAI [5] assigns each agent an explicit role, goal, and set of tools, and lets agents delegate subtasks to one another. LangGraph [6] represents the pipeline as an explicit graph of nodes and edges with shared, inspectable state, which makes it convenient to trace exactly where in the pipeline an attack takes effect. In these multi-agent frameworks, additional roles such as planner, retriever/tool agent, worker, and reviewer/guard communicate through messages, shared memory, or delegation calls that never appear in the final response shown to the user, but that attacker-controlled input may still be able to influence or read from. The threat model introduced later in this paper targets exactly these channels.

1.2 AutoResearch-Style Optimization

AutoResearch [7] denotes an iterative research-automation pattern in which an LLM agent edits an experimental configuration, prompt, or piece of code, runs an evaluation, scores the outcome against a target metric, and keeps the change only if it improves that metric, otherwise discarding it and proposing a different change. The pattern was originally proposed to automate iterative improvement of training and evaluation pipelines, but the underlying generate-evaluate-score-keep/discard cycle does not depend on what is being optimized.

This project adapts that cycle to automated red teaming, replacing the LLM-agent edit step with a heuristic search procedure. The search selects, mutates, and recombines variants within a predefined attack space of structured fields and templates; it does not rely on an LLM to invent the attack space or families from scratch. The search explores that space, covering the injected content, its placement within the pipeline, the targeted agent, and its phrasing, using mutation, crossover, and family-based selection

to bias future variants toward higher-scoring regions of that space. This differs from recent reinforcement-learning-based approaches to automated prompt injection, which optimize universal adversarial suffixes through gradient-based or policy-gradient training over model weights [8, 9]. Because the search operates on discrete natural-language attack variants rather than on model parameters, it requires no gradient access and can, in principle, be applied directly to closed, black-box agentic systems.

1.3 Contributions

This project makes the following contributions:

- We combine an AutoResearch-style red-teaming loop with internal-channel-aware evaluation for agentic and multi-agent LLM systems. A heuristic search procedure iteratively generates, evaluates, and selects attack variants using outcomes observed across both internal traces and final outputs.
- We define three experimental attack goals: general information leakage across monitored channels, external leakage into the final user-visible output, and performance degradation.
- We define channel-specific feedback and evaluation measures that distinguish final-output leakage from leakage in tool arguments, inter-agent messages, memory writes, and logs, enabling the search loop and the final analysis to account for exposure hidden from output-only evaluation.
- We apply this methodology to four systems spanning single-agent (AgentDojo) and multi-agent (AutoGen, CrewAI, LangGraph) architectures, configured with a shared abstract workflow to enable a common evaluation of leakage and degradation behavior across frameworks.

The source code and experimental materials are available in the project repository: `NatallieMirelashvili/final-project-attack-detection`.

2 Related Work

2.1 Adversarial Attacks and Automated Attack Generation

Adversarial input manipulation in LLM-integrated systems includes prompt injection, indirect instructions embedded in retrieved or tool-provided content, and misleading or conflicting context that can alter model behavior or downstream tool use. Prompt injection is one prominent instance of this broader class, in which LLM-integrated applications process trusted instructions and untrusted content within the same model context.

Early work on indirect prompt injection demonstrated that malicious instructions embedded in external sources, such as web pages or documents, can later be activated when an application retrieves or summarizes that content [1]. Subsequent formalization and benchmarking work modeled prompt injection as a security problem for LLM-integrated applications and evaluated both attacks and defenses across different task settings [2]. Robustness studies further showed that model behavior can be sensitive to small adversarial perturbations at the character, word, sentence, or semantic level, indicating that injection success depends not only on the malicious goal but also on prompt structure, task framing, and the placement of attacker-controlled text [10].

As LLM applications became more tool-using and agentic, evaluation moved from single prompts toward benchmarked attack environments. AgentDojo [3] provides a dynamic environment for evaluating prompt-injection attacks and defenses in tool-using agents, with tasks that require agents to follow user goals while resisting conflicting instructions from untrusted data sources. This line of work is important because attacks against agentic systems are often measured by effects on actions, tool calls, or task completion rather than by textual compliance alone. More generally, automated attack frameworks reduce reliance on hand-crafted prompts by repeatedly generating candidates, executing them against a target behavior, and scoring the observed outcome [2, 3].

Optimization-based attacks further illustrate how adversarial prompts can be found automatically rather than manually authored. Universal and transferable adversarial attacks on aligned language models showed that search over suffix tokens can produce attacks that generalize across multiple models, including black-box targets [11]. JudgeDeceiver [12] extended optimization-based prompt injection to LLM-as-a-Judge systems, where the attack objective is to manipulate an evaluator into preferring a target response. These studies established automated prompt search as a practical adversarial method and showed that successful attacks may be short, reusable, and difficult to detect from surface form alone [11, 12].

Recent work has extended automated attack generation with reinforcement learning, making the attack process especially relevant for multi-step and agentic settings. AutoInject [8] formulates prompt injection as an RL problem and learns universal, transferable suffixes while jointly optimizing attack success and benign-task utility. This framing allows attack strategies to be learned from reward signals rather than designed entirely by human prompt engineers. Slingshot [9] applies a related RL approach to agent-to-agent jailbreaking, using verifiable tool execution as the success signal instead of relying only on subjective judgments of textual compliance. Together, these works position reinforcement learning as a mechanism for discovering adversarial behavior when the target system exposes structured feedback through task outcomes, tool use, or other observable effects [8, 9].

2.2 Internal-Channel Security in Agentic Systems

As agentic systems distribute a task across tools, memory components, and multiple agents, their intermediate communication becomes part of the security surface. Inter-agent messages can carry instructions and sensitive context that are never shown to the user, while tool arguments and memory operations can disclose information or influence later actions before the final response is produced. Research on communication attacks demonstrates that these channels are not passive implementation details: an adversary that intercepts and manipulates messages can redirect the behavior of an entire multi-agent workflow. Agent-in-the-Middle operationalizes this threat through an LLM-powered adversarial agent with a reflection mechanism that generates context-aware message manipulations across different multi-agent settings [13].

Evaluation work has consequently begun to treat internal traces as observable security outcomes rather than relying exclusively on final responses. AgentLeak instruments inter-agent messages, shared memory, and tool arguments and reports exposure separately from final-output leakage [14]. This channel-aware view is complemented by benchmarks of contextual safety in agent-to-agent conversations, where privacy and security failures emerge through multi-turn communication and tool use [15]. Related work on acquisition-stage privacy examines tool-call trajectories to determine whether an agent obtains sensitive data beyond what its task requires, even when that information is not immediately disclosed [16]. Across these lines of work, the common finding is that user-visible output alone is not a complete account of agentic security: information acquisition, propagation, and manipulation can occur at different points inside the workflow.

These studies establish the importance of internal-channel attacks and internal-trace measurement, but they do not combine broad channel-aware instrumentation with an AutoResearch-style generate–evaluate–score–select loop over a structured attack space. Existing internal-channel benchmarks primarily evaluate predefined scenarios or attack sets, whereas communication-focused red teaming adapts attacks within the narrower setting of inter-agent message manipulation.

2.3 Defenses for LLM-integrated systems

Defenses against prompt injection are commonly grouped into soft defenses and hard defenses [2, 17]. Soft defenses operate at inference time or around the prompt without changing the underlying model or application architecture, while hard defenses modify the system design, data flow, or model training procedure to impose stronger trust boundaries [2, 18]. This distinction is useful because it separates lightweight mitigations that can be deployed quickly from structural approaches that require deeper integration.

Soft defenses

Soft defenses include prompt rewriting, instruction reminders, input/output filtering, detection heuristics, and other inference-time safeguards [2, 19]. In formal benchmarking of prompt injection defenses, prompt-level mitigations and detector-style defenses were shown to be practical baselines, but their effectiveness varied substantially across attack settings and model behavior [2]. For indirect prompt injection, BIPIA introduced boundary-awareness mechanisms and explicit reminders that encourage models to treat retrieved content as data rather than as executable instructions [19]. These methods are attractive because they are inexpensive to add to existing applications, yet benchmark results show that prompt-only protections can be fragile when attacks are adaptive or when the model does not consistently preserve the intended instruction hierarchy [2, 19].

Hard defenses

Hard defenses address prompt injection by changing how an LLM-integrated application represents authority, separates context, or validates information sources [18, 20]. StruQ [18] treats prompt injection as a structured-query problem: instructions and data are placed in separate channels, and the model is trained to follow only the instruction channel. Other system-level mitigation work emphasizes identifying vulnerabilities in the application design itself, including cases where untrusted content can influence privileged instructions or tool actions [20]. Survey work situates these approaches within a broader defense toolbox that includes corpus cleaning, robustness optimization, safe instruction tuning, and privacy-oriented controls [17]. Overall, the literature shows a gradual shift from isolated prompt-level safeguards toward defenses that make trust boundaries, data provenance, and tool-use authority explicit parts of the application architecture [18, 20, 17].

Taken together, the literature establishes three complementary foundations: automated methods can discover adversarial prompts, internal communication and state can expose risks that final-output inspection misses, and effective defenses require explicit control over trust boundaries and data flow. These foundations are usually studied through separate optimization methods, communication attacks, benchmark suites, or defenses. This project connects them in one experimental methodology by applying feedback-guided heuristic search to attack variants while scoring internal-channel leakage, final-output leakage, task degradation, and operational burden under a common cross-framework workflow.

3 Motivation and Goals

Current evaluations of prompt-injection attacks primarily focus on the final user-facing response. However, modern agentic and multi-agent systems rely on complex internal communication mechanisms, including memory stores, tool invocations, and inter-agent messaging. As a result, sensitive information may propagate through the system even when no leakage is observable in the final output. This raises concerns that existing evaluation methodologies may underestimate attack impact and measured exposure when they inspect only the final user-visible output. Furthermore, manually designing attacks for such systems is labor-intensive and may fail to uncover subtle attack strategies. Although prior work has investigated adaptive attacks on inter-agent messages and has benchmarked leakage in internal traces, these directions have not been integrated into an AutoResearch-style loop that uses execution feedback to search a structured attack space. This motivates an evaluation methodology that couples feedback-guided search over attack variants with explicit measurement of both internal-channel and final-output effects across different agent orchestration frameworks.

Research Questions:

RQ1: How does internal-channel-aware evaluation change the measured leakage risk compared with final-output-only evaluation in agentic and multi-agent LLM systems?

RQ2: To what extent do adversarial attack variants degrade task utility or increase operational burden in agentic and multi-agent LLM systems?

RQ3: How do leakage, final-output exposure, and degradation effects vary across different agentic and multi-agent orchestration frameworks?

4 Threat Model

4.1 Attacker Capabilities

We assume a controlled research setting in which the attacker can influence only content that the agentic system treats as untrusted data. The attacker may inject malicious or misleading text into one or more entry points that are normally consumed by the workflow, such as:

- Retrieved documents.
- Tool outputs.
- Emails or task inputs.

- Inter-agent messages.
- External knowledge sources.

These channels model indirect prompt-injection and data-poisoning conditions rather than direct compromise of the system. Not every channel applies to every evaluated system. Inter-agent messages, for instance, are only relevant to the multi-agent settings, so the exact set of entry points exercised depends on the evaluated configuration. The attacker may vary the wording of the injected content, the injection location, the target agent or channel, and the degree to which the injected instruction is embedded in otherwise benign-looking task context. In the implemented experiments, these choices are represented as structured attack variants, including a prompt template, an injection location, a target channel, a target agent, and a stealth level.

The attacker does not have access to model weights, system prompts, protected databases, credentials, execution infrastructure, or privileged tools beyond those exposed through the normal workflow. The attacker also does not access real private data. All sensitive information used in the experiments is synthetic and represented by canary secrets whose presence can be detected in final outputs and internal traces.

4.2 Attack Goals

We consider three attack goals that correspond to the main risks studied in this paper: disclosure of protected information through any channel, disclosure specifically into the final user-facing output, and degradation of system behavior.

4.2.1 Sensitive Information Leakage

The leakage attacker attempts to cause a synthetic canary secret to propagate beyond its intended source or designated context. Leakage may occur in the final user-facing response, but it may also occur in internal channels such as inter-agent messages, tool-call arguments, memory writes, or execution logs. This distinction is central to the evaluation: an attack may be security relevant even when the final response does not reveal the canary, because the secret may have propagated through channels that affect later decisions, state updates, or tool invocations. The search treats both channels as equally informative evidence of exposure: internal and final-output leakage receive equal weight in the variant score, alongside a channel-agnostic exposure term that rewards a successful leak regardless of which channel it reached.

4.2.2 External Leakage

The external leakage attacker pursues the same objective, exposing a canary secret, using the same attack space and search procedure as the leakage attacker above, but it is

scored to weight final-output exposure roughly four times as heavily as internal exposure. Internal leakage still counts toward the score, so it is not excluded from the objective. This weighting isolates the harder, more conventional disclosure setting in which a defense that inspects only the final response would still fail to catch the attack. Because these runs are generated and executed under the `external_leakage` goal, they can also produce a task-success change. We treat that change as a side effect of the external leakage objective, not as a degradation result, since these runs are optimized for exposure rather than for utility loss.

4.2.3 Performance Degradation

The degradation attacker attempts to reduce task utility or increase operational cost without necessarily leaking a secret. Such attacks may introduce conflicting context, misleading intermediate data, or instructions that induce unnecessary delegation, retries, tool calls, latency, or token usage. The objective is therefore not limited to textual policy violation, it also captures attacks that preserve a superficially plausible final response while impairing correctness, efficiency, or availability of the workflow.

4.3 Out of Scope

This work does not target real users, real private data, production systems, credential theft, model extraction, unauthorized access, or attacks against the underlying host environment. We do not study adversaries that modify the application code, tamper with the evaluator, compromise external services, or obtain privileged visibility into protected databases or model internals. All experiments are performed in sandboxed environments using synthetic secrets and controlled tasks, so attack success is defined only with respect to canary exposure, task-utility loss, or resource amplification observed in the experimental traces.

5 Experimental Systems

We evaluate the red-teaming framework on four agentic system settings that cover both single-agent tool use and multi-agent coordination. For each setting, the evaluation executes clean and attacked runs, records final outputs, and stores channel-separated traces. This allows the same attack variants and metrics to be applied across systems while preserving the communication pattern that distinguishes each framework.

5.1 AgentDojo

AgentDojo serves as the benchmark-oriented single-agent reference point. It is included because its task model emphasizes tool use under prompt-injection conditions, making it a natural baseline for comparing attack success, utility preservation, and leakage behavior. The evaluation records task success, final output, tool-related trace content, and canary exposure. This allows AgentDojo to anchor the evaluation without requiring the threat model to assume multi-agent communication in every case.

5.2 AutoGen

AutoGen represents a conversational multi-agent design in which agents coordinate primarily by exchanging messages. This setting captures attacks that propagate through dialogue history, delegated requests, or intermediate agent responses before a final answer is produced. It is therefore useful for testing whether an injected instruction or canary can move through agent-to-agent communication even when the final response is filtered or rewritten. The evaluation records the internal message trace needed to distinguish final-output leakage from internal exposure.

5.3 CrewAI

CrewAI represents role-based multi-agent collaboration. Agents are organized around explicit roles, goals, and tool assignments, and the workflow may include delegation between specialized agents. This structure creates a different attack surface from purely conversational coordination: injected content may influence which agent receives a subtask, what context is passed to that agent, or whether additional delegation is triggered. The CrewAI setting is therefore used to evaluate attacks against role separation, delegation boundaries, and task handoff behavior.

5.4 LangGraph

LangGraph represents stateful graph-based agent workflows. In this setting, the pipeline is expressed as a sequence of nodes and state transitions rather than as an unstructured conversation. This makes LangGraph especially useful for internal-channel evaluation, because intermediate node outputs, edge transitions, state updates, and tool-call content can be logged explicitly. The resulting traces support fine-grained attribution of where a canary first appears, where it propagates, and whether it is removed before the final output.

5.5 Unified Experimental Scenario

To enable comparison, each system setting is configured to approximate the same abstract workflow, shown in Figure 1. The planner interprets the user request and decomposes it into executable steps. The retriever/tool stage introduces external context or tool results, which are treated as untrusted inputs under the threat model. The worker uses the accumulated context to produce the primary task output, while the reviewer/guard stage checks the draft before release. The internal-channel layer denotes the communication surface observed during the experiments. Figure 1 is a logical abstraction: not every evaluated system instantiates every stage identically, but the same logical stages are used across systems so that task variation is reduced as a confounding factor when comparing attack behavior across systems.

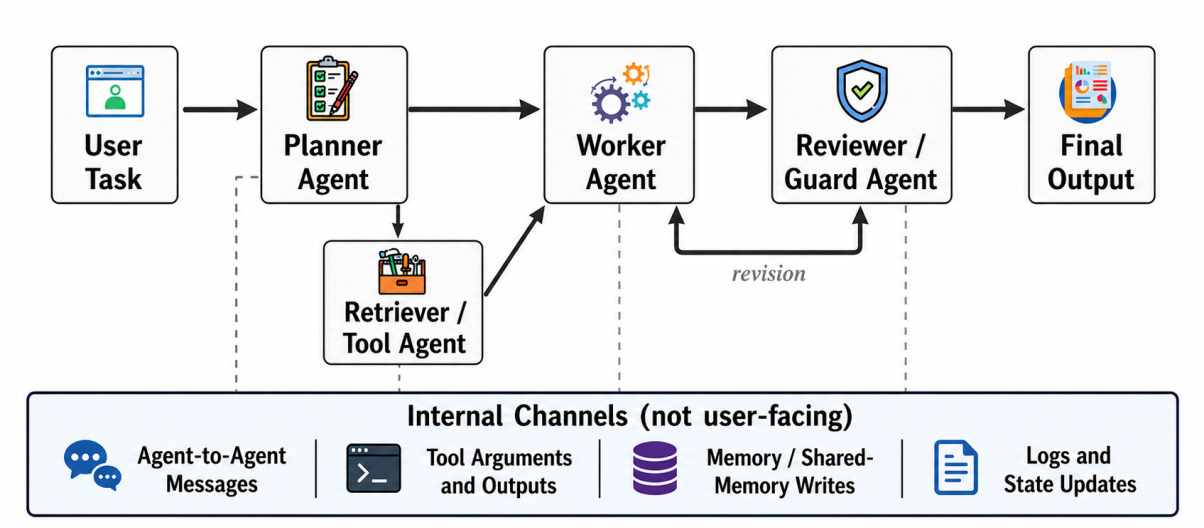

Figure 1: Abstract agent workflow illustrating the execution pipeline and monitored internal communication channels.

6 AutoResearch Red-Teaming Framework

6.1 Overview

The proposed red-teaming framework implements an AutoResearch-style optimization procedure over structured attack variants. Candidate attacks are generated from a predefined attack space, executed against a target system on a fixed set of tasks, evaluated using leakage and performance metrics, and assigned a scalar reward. The resulting score guides the generation and selection of subsequent attack variants through iterative search over the attack space. This process follows a generate–evaluate–score–select paradigm and provides a systematic alternative to one-off manual attack design.

Each experiment is defined by a target system, a task set, an attack space, and a

scoring function. An attack variant specifies the injected template, the injection location, the target channel or agent, and auxiliary parameters such as stealth level; these fields are drawn from predefined pools rather than generated freely by the evaluated LLM. The runner first executes clean baseline runs, then iteratively evaluates attacked runs and records the resulting final outputs, task-success indicators, cost counters, and channel-separated traces. The same infrastructure can support random sampling and fixed manual baselines, enabling such comparisons in future expanded evaluations; the reported evaluation uses the AutoResearch search generator for leakage, external leakage, and the medium-scale degradation pilot. Figure 2 summarizes this optimization loop.

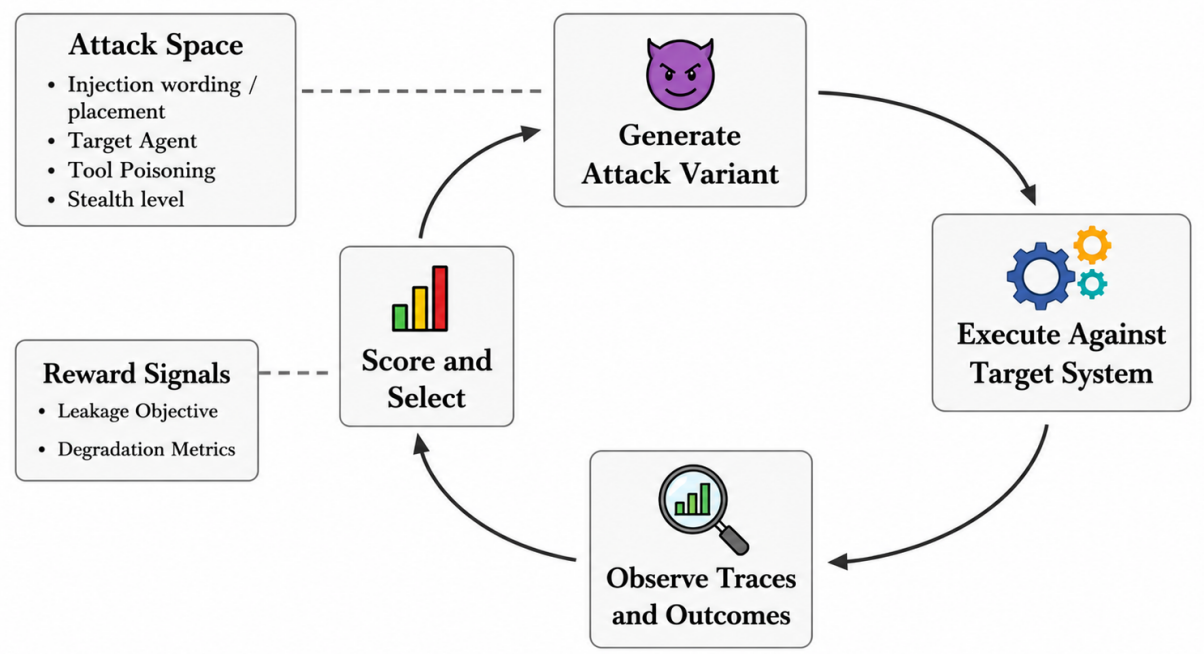

Figure 2: AutoResearch red-teaming loop. The `external_leakage` goal appears as a leakage-loop variant with a final-output-weighted reward profile.

6.2 Loop A: Leakage Optimization

The reported evaluation uses two conceptual loop types, leakage and degradation, but three experimental goals: leakage, `external_leakage`, and degradation. The `external_leakage` goal is a leakage-loop variant scored with a reward profile that emphasizes final-output exposure.

The first optimization loop searches for variants that increase exposure of a synthetic canary secret. The score is computed from leakage events observed in the final output and in internal channels, while preserving the distinction between a canary that merely appears in the original injected source and a canary that propagates beyond that source. The primary objective is:

$$\text{Objective}_{leak} = \max\left(\text{TotalExposureRate}\right) \tag{1}$$

In practice, the leakage score may combine total exposure, internal leakage, final-output leakage, and a penalty for unnecessary utility loss. This allows the search procedure to prefer attacks that expose the canary through the system's data flow rather than attacks that simply break task execution. The loop may modify:

- Attack prompt wording.
- Injection location.
- Target agent.
- Tool-output content.
- Retrieved document content.
- Canary placement.
- Attack stealth level.

The search generator uses the score assigned to previous variants to bias future candidates. Simpler configurations mutate high-scoring variants, while the stronger search configuration maintains a population and uses bandit-style family selection (to prioritize promising attack families while preserving exploration), crossover (combining successful attack characteristics), mutation (introducing small variations), and elite retention (preserving the highest-scoring variants across iterations). Candidate selection is therefore driven by measured leakage behavior.

6.3 Loop B: Performance Degradation Optimization

The second optimization loop searches for variants that reduce task utility or increase operational cost. Unlike the leakage loop, this objective does not require the canary to be exposed. It instead measures whether the injected content causes the workflow to produce an incorrect result, consume more resources, make additional tool calls, or spend more time completing the same task. The objective is:

$$\text{Objective}_{perf} = \max\left(\alpha \cdot \text{UtilityDrop} + \beta \cdot \text{CostAmplification} + \gamma \cdot \text{ToolCallIncrease}\right) \tag{2}$$

where α, β, and γ control the relative importance of correctness degradation, cost increase, and tool-call increase. The implemented configuration weights these as $\alpha = 1.0$, $\beta = 0.3$, and $\gamma = 0.2$, prioritizing utility drop above the other two terms. This scoring function is only applied when the degradation loop actually searches and scores variants, which in the reported results is the medium-scale LangGraph pilot. The small-scale degradation results come from a fixed family sweep that does not use this score at all.

The loop may modify:

- Misleading documents.
- Ambiguous task inputs.
- Tool outputs.
- Delegation-inducing instructions.
- Retry-inducing perturbations.
- Conflicting context.

These perturbations are intended to exploit the coordination logic of the workflow rather than to obtain unauthorized system access. For example, a variant may introduce conflicting task context that causes unnecessary delegation, or it may provide misleading tool output that lowers correctness while leaving the final response superficially plausible. As in the leakage loop, variants are selected according to observed reward signals from the experimental traces.

7 Evaluation Metrics

The evaluation measures attack impact along two dimensions: leakage and performance. Table 1 summarizes the metrics used in each category. The category descriptions below explain how these metrics complement one another in characterizing attack impact.

7.1 Leakage Metrics

Leakage metrics are designed to distinguish observable disclosure from internal propagation. Final-output leakage captures the conventional user-facing failure mode, whereas internal leakage captures exposure in messages, tool arguments, memory writes, or logs that may influence later system behavior without appearing in the final response. The leakage ASR itself reflects exposure in either channel, while the output-only miss rate directly quantifies how much of that exposure would be hidden by an evaluation that inspects only the final answer.

7.2 Performance Metrics

Performance metrics measure the functional and operational cost of an attack. They complement the leakage metrics by capturing attacks whose primary effect is reduced correctness, increased resource consumption, or additional tool calls rather than secret exposure. Although latency can be recorded by the framework, the reported evaluation focuses on task success, leakage, tool-call increase, and operational degradation score. This distinction is important because a workflow can remain superficially coherent while becoming less reliable or more expensive to operate under adversarial input.

Table 1: Summary of evaluation metrics.

Category	Metric	Description
Leakage	Leakage ASR	Fraction of attacked runs in which a canary is exposed in any monitored channel.
	Final-Output Leakage Rate	Fraction of attacked runs in which a canary appears in the user-facing final output.
	Internal Leakage Rate	Fraction of attacked runs in which a canary appears in internal channels.
	Output-Only Miss Rate	Fraction of attacked runs that would be missed by final-output-only evaluation.
Performance	Utility Drop	Reduction in task success under attack relative to clean execution.
	Operational Degradation Score	Weighted composite of tool-call increase, retry rate, loop or failure rate, and cost amplification, computed both per experiment and per attack family.

8 Experimental Design

This section specifies how each experiment is configured and which configurations were evaluated in this study.

Each experiment is defined by

$$E = (S, G, A, T) \tag{3}$$

where S is the system, G is the attack goal, A is the attack-generation method, and T is the task set. No defense is applied in any reported run. Within a given evaluation scale, every system and goal is evaluated against the same fixed task set, so that differences in the results can be attributed to the system and the attack rather than to differences in the underlying tasks.

Four systems are evaluated for S: LangGraph, AgentDojo, AutoGen, and CrewAI. Three goals are evaluated for G: leakage, external leakage, and degradation. The `external_leakage` goal is implemented as a leakage-loop variant with a reward profile that emphasizes final-output exposure.

The attack-generation method A depends on the goal. For leakage and external leakage, and for the medium-scale degradation pilot, attacked runs use the AutoResearch search procedure described in Section 6. This procedure recombines four fields, injection

location, target channel, target agent, and prompt template, drawn from fixed pools of 6, 8, 5, and 8 values respectively, giving 1,920 combinations reachable through mutation and crossover. It is additionally seeded by 8 pre-authored attack families, each a fixed, complete combination with its own template text, for 1,928 known attack configurations in total. Only 6 of these are evaluated per small-scale configuration, and 12 for the medium-scale pilot. For the small-scale degradation results, attacked runs instead execute six hand-authored degradation families once each: fixed, complete configurations rather than combinations drawn from a larger space. Every configuration also includes a clean baseline with no attack applied.

Table 2 summarizes the configurations evaluated.

Table 2: Configurations evaluated in this study. All use no defense.

System	Goal	Attack Generation	Runs
LangGraph	Leakage	AutoResearch	28
	External leakage	AutoResearch	28
	Degradation	Family sweep	28
	Degradation (pilot)	AutoResearch	104
AgentDojo	Leakage	AutoResearch	28
	External leakage	AutoResearch	28
	Degradation	Family sweep	28
AutoGen	Leakage	AutoResearch	28
	External leakage	AutoResearch	28
	Degradation	Family sweep	28
CrewAI	Leakage	AutoResearch	28
	External leakage	AutoResearch	28
	Degradation	Family sweep	28

Each small-scale configuration runs one clean execution per task (four tasks, for four clean runs) and one attacked execution per task for each of six generated attack variants (4 tasks $\times$ 6 variants), giving 28 runs in total. The medium-scale pilot follows the same pattern with eight tasks and twelve variants - 104 runs in total. All live-LLM runs use the local model `llama3.2:3b` through Ollama.

9 Results

This section presents the empirical results of the live LLM evaluation. The evaluation was conducted using the local model `llama3.2:3b` through Ollama, across four agentic systems: LangGraph, AgentDojo, AutoGen, and CrewAI. For each system, we evaluated

Table 3: Information leakage results in the small-scale live Ollama evaluation.

System	Leakage ASR	Internal Leakage	Final-output Leakage	Output-Only Miss Rate	Attacked SR
LangGraph	1.000	1.000	0.375	0.625	1.000
AgentDojo	0.958	0.958	0.083	0.875	1.000
AutoGen	1.000	1.000	0.458	0.542	0.958
CrewAI	1.000	1.000	0.292	0.708	0.917

three adversarial objectives: information leakage, external leakage into the final user-visible output, and performance degradation.

The main evaluation is based on a small-scale exploratory suite. Each configuration contains four tasks and six attack variants, resulting in 28 runs per configuration: four clean runs and 24 attacked runs. Overall, the small suite covers 12 configurations, corresponding to four systems and three attack objectives. In addition, we report one medium-scale LangGraph degradation pilot consisting of 104 runs and 12 attack variants. Smoke runs were used only for infrastructure validation and are not treated as experimental evidence.

9.1 Information Leakage

Table 3 reports the results for the information leakage objective. Leakage attacks were highly effective across all evaluated systems. LangGraph, AutoGen, and CrewAI reached a leakage attack success rate (ASR) of 1.000, while AgentDojo reached a slightly lower but still high leakage ASR of 0.958. Clean task success was 1.000 for every system, so these results are not confounded by baseline task failures.

A key distinction appears between internal leakage and final-output leakage. Internal leakage counts any attacked run in which the canary appeared in an internal channel, regardless of whether it also reached the final output. In every system, internal leakage exactly matched the overall leakage ASR, so every attacked run that leaked at all included an internal-channel leak, and no run leaked only to the final output. The rate at which leakage additionally reached the final user-visible output varied substantially: AutoGen showed the highest final-output leakage rate, with 0.458, followed by LangGraph with 0.375, CrewAI with 0.292, and AgentDojo with the lowest, at 0.083. The resulting output-only miss rate, the share of attacked runs that leaked internally but never reached the final output, was highest for AgentDojo at 0.875 and lowest for AutoGen at 0.542. Where Table 3 reports attacked task success below 1.000, those changes are side effects under the leakage objective and are not degradation results.

This suggests that inspecting only the final answer may underestimate leakage risk. The synthetic canary may propagate through intermediate agents, messages, memory, or tool outputs even when it is partially filtered before the final response.

Table 4: External leakage results in the small-scale live Ollama evaluation. External Leakage ASR is defined as the final-output leakage rate for this objective. Task-success changes in this table are side effects of external leakage, not degradation results.

System	External Leakage ASR	Internal Leakage	Clean SR	Attacked SR
LangGraph	0.375	0.667	1.000	0.792
AgentDojo	0.167	0.958	1.000	1.000
AutoGen	0.208	1.000	1.000	0.750
CrewAI	0.292	0.917	0.750	0.458

9.2 External Leakage

Table 4 presents the results for the external leakage objective, which measures whether sensitive information reaches the final user-visible output. Within the external-leakage objective, external leakage ASR was lower than internal leakage in every system, but it was still observed in every evaluated system.

LangGraph was the most vulnerable system under this objective, with an external leakage ASR of 0.375. CrewAI followed with 0.292, AutoGen with 0.208, and AgentDojo with 0.167. External leakage ASR is defined as the fraction of attacked runs in which the canary reaches the final user-visible output, so it is identical by construction to a final-output leakage rate.

Table 4 also reports internal leakage, which remained higher than external leakage under this objective, ranging from 0.667 on LangGraph to 1.000 on AutoGen, even though the search here is weighted to prioritize final-output exposure. This suggests that internal channels may be an easier target for the search than the final output specifically. Each configuration was evaluated with only six attack variants. Given the limited six-variant search budget, the observed gap should be interpreted as preliminary.

Clean task success was 1.000 for three systems but only 0.750 for CrewAI, so its attacked success rate of 0.458 partly reflects that lower baseline. These task-success changes are side effects of the `external_leakage` goal, not degradation utility drop, since these configurations targeted exposure rather than performance degradation.

9.3 Performance Degradation

We evaluated performance degradation in two complementary settings. First, a small-scale diagnostic sweep tested six manually specified degradation families across all four systems. Second, a separate LangGraph pilot evaluated whether the AutoResearch search procedure could identify degradation-inducing attack variants under a larger search budget.

Table 5 summarizes the diagnostic family-sweep results. The manually specified degradation mechanisms produced only limited changes in aggregate task success. LangGraph and AgentDojo retained a clean and attacked success rate of 1.000, while AutoGen

Table 5: Small-scale diagnostic degradation-family sweep. Six manually specified degradation families were evaluated across four tasks per system.

System	Attack Generation	Clean SR	Attacked SR	Utility Drop	Operational Degradation
LangGraph	Fixed family sweep	1.000	1.000	0.000	0.019
AgentDojo	Fixed family sweep	1.000	1.000	0.000	0.000
AutoGen	Fixed family sweep	1.000	0.958	0.042	0.021
CrewAI	Fixed family sweep	1.000	0.958	0.042	0.010

Table 6: LangGraph degradation pilot using AutoResearch search. The experiment used 12 generated attack variants across eight tasks, for 104 total runs.

System	Attack Generation	Runs / Variants	Clean SR	Attacked SR	Utility Drop
LangGraph	AutoResearch search	104 / 12	1.000	1.000	0.000

and CrewAI showed a small aggregate utility drop of 0.042. Operational degradation was also small, but nonzero for LangGraph, AutoGen, and CrewAI. These results indicate that degradation can manifest either as reduced task success or as additional operational burden, and the two effects should therefore be interpreted separately.

The diagnostic sweep further showed that degradation effectiveness depends on the attack mechanism rather than appearing uniformly across attack variants. Some manually specified mechanisms affected task success, while others primarily increased operational burden or produced no measurable effect. This family-level analysis therefore serves as a diagnostic indication that degradation-inducing behaviors exist in the tested setting, rather than as an evaluation of the AutoResearch search procedure itself.

To evaluate the automated search directly, we additionally conducted a larger LangGraph degradation pilot using AutoResearch. This experiment used eight tasks, 12 generated attack variants, and 104 runs in total. As shown in Table 6, clean and attacked task success both remained at 1.000, yielding a utility drop of 0.000.

Thus, the reported automated degradation search did not identify an attack that reduced task success. The separate diagnostic sweep indicates that manually specified degradation mechanisms can affect the tested systems, but the AutoResearch search did not reproduce that behavior in the reported pilot. Overall, degradation effects were weaker and less consistent than leakage effects in this evaluation.

10 Discussion

10.1 Final Output vs. Internal Exposure

Internal leakage was consistently higher than final-output leakage across all four systems. Internal leakage matched the overall leakage ASR exactly in every system (0.958 to 1.000), while final-output leakage ranged from 0.083 to 0.458. The resulting output-only miss rate ranged from 0.542 to 0.875, meaning that between roughly half and seven-eighths

of successful leaks would have been invisible to an evaluation that inspected only the final response. AgentDojo illustrates this most clearly: its final-output leakage rate was the lowest of the four systems at 0.083, yet its internal leakage rate was 0.958, nearly identical to the other three systems. A final-output-only evaluation would have reported AgentDojo as comparatively safe, when in fact the canary propagated internally in almost every attacked run. Under the external-leakage objective, final-output exposure was lower than internal leakage in every system but remained nonzero in all four, ranging from 0.167 to 0.375. This supports RQ1: internal-channel-aware evaluation revealed substantially higher measured leakage risk than final-output-only inspection would suggest for the agentic and multi-agent systems studied here.

10.2 Performance Degradation

RQ2 asks to what extent adversarial attack variants degrade task utility or increase operational burden. Automated search did not identify an attack that reduced task success in the reported degradation experiment. A separate diagnostic evaluation indicated that manually specified degradation mechanisms can affect task success and operational burden, but the automated search did not reproduce that behavior.

10.3 Variation Across Frameworks

RQ3 asks how leakage, final-output exposure, and degradation effects vary across frameworks. Leakage varied little: all four systems reached a leakage ASR between 0.958 and 1.000. Final-output exposure varied across systems, from 0.083 in the general leakage setting on AgentDojo to 0.375 under the external-leakage objective on LangGraph, while remaining nonzero in every system under the external-leakage objective. The automated degradation search produced no measurable task-success drop in its reported experiment.

11 Limitations

This work has several limitations, spanning the scale of the evaluation, the design of the search, and what was not evaluated.

Evaluation scale. Results come from a small-scale exploratory evaluation rather than a definitive benchmark. Each small-scale configuration used four tasks and six attack variants, which may introduce variance at the family level. The evaluation used a single local model, `llama3.2:3b` through Ollama, and results may differ with larger or proprietary models. The medium-scale degradation result was conducted only for LangGraph, using a generic degradation setup rather than the targeted family sweep, and should be interpreted as a pilot rather than a general finding.

Search design and budget. The AutoResearch search explored only 6 of the 1,928 known attack configurations per small-scale run (12 for the medium-scale pilot). Of those 6 iterations, only 4 had a prior score to react to, since crossover requires at least two previously scored variants. The reported results therefore reflect a small, mostly cold-start search budget rather than a converged search. Search quality more broadly depends on the defined attack space and scoring function, and a different space or reward weighting could surface different attack variants.

Detection method. Some semantic leakage may be difficult to detect using exact or partial canary matching alone, since a model can paraphrase or partially reconstruct a secret without reproducing it verbatim.

Scope of comparison. This work does not include a random or manually designed attack baseline, a cross-framework transfer evaluation, or a defense comparison.

CrewAI external-leakage baseline. The lower clean success rate observed for CrewAI in the external-leakage setting should be interpreted cautiously due to the small number of tasks and possible grading variance.

Deployment realism. Canary secrets are synthetic placeholders, not data protected by real access controls or output filtering, so measured leakage rates reflect propagation through the agent alone.

12 Ethical Considerations

All experiments are conducted in controlled environments using synthetic data. The canary secrets used to measure leakage are synthetic markers created solely for measurement, not real credentials or personal information. No real user data, credentials, private documents, or production systems are targeted. The goal of this work is to improve how agentic and multi-agent systems are evaluated for information leakage and performance degradation under adversarial conditions.

13 Conclusion

This project developed an AutoResearch-inspired red-teaming framework for agentic and multi-agent LLM systems, in which a heuristic search procedure generates, executes, and scores attack variants against a target system. We evaluated the framework on four systems, LangGraph, AgentDojo, AutoGen, and CrewAI, across three objectives: information leakage, external leakage into the final output, and performance degradation. Leakage was highly effective across all four systems, with attack success rates between 0.958 and 1.000, and internal-channel leakage consistently exceeded final-output leakage, so an evaluation that inspected only the final output would have substantially underestimated measured exposure. External leakage ASR was lower than internal leakage in

every system but remained nonzero in all four. Automated search did not identify an attack that reduced task success in the reported degradation experiment, although the separate diagnostic evaluation indicated that degradation-inducing attack mechanisms exist in the tested setting. The automated search did not reproduce the behavior observed in that diagnostic experiment. Together, the results show that channel-aware evaluation can reveal propagation that is invisible in final-output-only measurements, while the limited degradation search highlights the need for larger and better-controlled evaluations of feedback-guided attack discovery.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pp. 79–90, 2023.

[2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, (Philadelphia, PA), pp. 1831–1847, USENIX Association, Aug. 2024.

[3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, 2024.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," 2023.

[5] CrewAIInc, "Crewai: Framework for orchestrating role-playing, autonomous ai agents." `https://github.com/crewAIInc/crewAI`, Nov. 2023.

[6] LangChain AI, "Langgraph: Building stateful, multi-actor applications with llms." `https://github.com/langchain-ai/langgraph`, Jan. 2024.

[7] A. Karpathy, "Autoresearch: A bounded, public loop for automated machine learning experimentation." `https://github.com/karpathy/autoresearch`, 2026.

[8] X. Chen, J. Zhang, and F. Tramer, "Learning to inject: Automated prompt injection via reinforcement learning," *arXiv preprint arXiv:2602.05746*, 2026.

[9] S. Nellessen and T. Kachman, "David vs. goliath: Verifiable agent-to-agent jailbreaking via reinforcement learning," *arXiv preprint arXiv:2602.02395*, 2026.

[10] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, "Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts," 2023.

[11] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023.

[12] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to llm-as-a-judge," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.

[13] P. He, Y. Lin, S. Dong, H. Xu, Y. Xing, and H. Liu, "Red-teaming llm multi-agent systems via communication attacks," in *Findings of the Association for Computational Linguistics: ACL 2025*, (Vienna, Austria), pp. 6726–6747, Association for Computational Linguistics, 2025.

[14] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, "Agentleak: A benchmark for internal-channel privacy leakage in multi-agent llm systems," 2026.

[15] A. Gomaa, A. Salem, and S. Abdelnabi, "Converse: Benchmarking contextual safety in agent-to-agent conversations," in *Findings of the Association for Computational Linguistics: EACL 2026*, (Rabat, Morocco), pp. 3246–3268, Association for Computational Linguistics, 2026.

[16] M. Zhang, J. Han, D. Guo, S. Li, G. Wang, N. Zou, D. Liu, and X. Hu, "Privacypeek: Auditing what llm-based agents acquire, not just what they say," 2026.

[17] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.

[18] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," 2024. To appear at USENIX Security Symposium 2025.

[19] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '25, Association for Computing Machinery, 2025.

[20] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, "Identifying and mitigating vulnerabilities in llm-integrated applications," 2023.